

CO4-Assessment Tool-03

CSA0613-Design & Analysis of Algorithms

Name:A.Bhavya(192424417)

3. MST Comparison (Prim vs Kruskal)

Given a weighted graph with multiple edges, construct the Minimum Spanning Tree using both Prim's and Kruskal's algorithms. Compare their edge selection strategies and intermediate results. Evaluate their performance in dense and sparse graphs. Analyze differences in implementation and derive conclusions on their suitability.

AIM

To construct the Minimum Spanning Tree (MST) of a weighted graph using Prim's and Kruskal's algorithms and compare their edge selection strategies, intermediate results, performance, and suitability for different types of graphs.

OBJECTIVE

- To construct an MST using Prim's algorithm.
- To construct an MST using Kruskal's algorithm.
- To compare the edge selection strategies of both algorithms.
- To analyze their performance in dense and sparse graphs.
- To study their time complexity and practical applications.

INTRODUCTION

A Minimum Spanning Tree (MST) is a subset of edges in a connected weighted graph that connects all vertices with the minimum possible total edge weight without forming any cycles.

Prim's and Kruskal's algorithms are popular greedy algorithms used to find the Minimum Spanning Tree.

Although both algorithms produce an MST, their method of selecting edges is different.

- Prim's algorithm starts from a vertex and grows the tree by selecting the minimum-weight edge connected to the current tree.
- Kruskal's algorithm sorts all edges and selects the minimum-weight edges while avoiding cycles.

GIVEN WEIGHTED GRAPH

Consider the following weighted edges:

Edge	Weight
A – B	2
A – C	3
B – C	1
B – D	4
C – D	5
C – E	6
D – E	2

The graph contains five vertices:

A, B, C, D, and E

MINIMUM SPANNING TREE USING PRIM'S ALGORITHM

Prim's algorithm starts with any vertex. Let us start from vertex A.

Step 1

Start with vertex:

A

Select the minimum edge connected to A:

$$A - B = 2$$

Step 2

Current vertices:

A, B

Available minimum edge:

$$B - C = 1$$

Select:

$$B - C = 1$$

Step 3

Current vertices:

A, B, C

Select the minimum edge connecting a new vertex:

$$B - D = 4$$

Step 4

Current vertices:

A, B, C, D

Select:

$$D - E = 2$$

MST Using Prim's Algorithm

Selected edges:

- $A - B = 2$
- $B - C = 1$
- $B - D = 4$
- $D - E = 2$

Total Weight:

$$2 + 1 + 4 + 2 = 9$$

MINIMUM SPANNING TREE USING KRUSKAL'S ALGORITHM

Kruskal's algorithm first sorts all edges according to their weights.

Sorted Edges

Edge	Weight
B – C	1
A – B	2
D – E	2
A – C	3
B – D	4
C – D	5
C – E	6

Step 1

Select:

$$B - C = 1$$

Step 2

Select:

$$A - B = 2$$

Step 3

Select:

$$D - E = 2$$

Step 4

Consider:

$$A - C = 3$$

This edge creates a cycle:

$$A \rightarrow B \rightarrow C \rightarrow A$$

Therefore, it is rejected.

Step 5

Select:

$$B - D = 4$$

Now all vertices are connected.

MST Using Kruskal's Algorithm

Selected edges:

- $B - C = 1$

- $A - B = 2$
- $D - E = 2$
- $B - D = 4$

Total Weight:

$$1 + 2 + 2 + 4 = 9$$

COMPARISON OF PRIM'S AND KRUSKAL'S ALGORITHMS

Feature	Prim's Algorithm	Kruskal's Algorithm
Approach	Starts from a vertex	Starts from edges
Edge Selection	Minimum edge connected to tree	Globally smallest edge
Cycle Checking	Not usually selected if outside tree	Uses cycle detection
Data Structure	Priority Queue / Min Heap	Disjoint Set / Union-Find
Best For	Dense graphs	Sparse graphs

INTERMEDIATE RESULTS COMPARISON

In Prim's algorithm, the tree grows continuously from a selected starting vertex.

Example:

$$A \rightarrow B \rightarrow C \rightarrow D \rightarrow E$$

In Kruskal's algorithm, edges can be selected from different parts of the graph initially.

Example:

- B – C
- A – B
- D – E

Later, these separate components are connected.

Thus, Prim's algorithm focuses on growing one connected tree, while Kruskal's algorithm focuses on selecting globally minimum edges without cycles.

Prim's Algorithm – Steps

1. Start with any vertex.
2. Mark the starting vertex as visited.
3. Find the minimum-weight edge connecting a visited vertex to an unvisited vertex.
4. Add that edge to the MST.
5. Mark the new vertex as visited.
6. Repeat until all vertices are included.
7. Calculate the total weight of the MST.
8. Display the selected edges and total weight.
9. Stop.

Prim's Algorithm – Python Code

```

main.py Visualize Run
21 while priority_queue:
24     if current in visited:
25         continue
26
27     visited.add(current)
28
29     if parent is not None:
30         mst.append((parent, current, weight))
31         total_cost += weight
32
33     for neighbor, edge_weight in graph[current]:
34         if neighbor not in visited:
35             heapq.heappush(
36                 priority_queue,
37                 (edge_weight, neighbor, current)
38             )
39
40 print("Minimum Spanning Tree using Prim's Algorithm:\n")
41
42 for edge in mst:
43     print(f"{edge[0]} - {edge[1]} : {edge[2]}")
44
45 print("\nTotal Weight of MST:" total_cost)

```

Output

```

Programiz Python Online Compiler
Trade on MT4 with Raw Pricing Ultra-low latency execution Start Trading
Trading derivatives involves high risk to your capital.

main.py Visualize Run
4 graph = {
5     'A': [('B', 2), ('C', 3)],
6     'B': [('A', 2), ('C', 1), ('D', 4)],
7     'C': [('A', 3), ('B', 1), ('D', 5), ('E', 6)],
8     'D': [('B', 4), ('C', 5), ('E', 2)],
9     'E': [('C', 6), ('D', 2)]
10 }
11
12 start = 'A'
13
14 visited = set()
15 mst = []
16 total_cost = 0
17
18 # Priority queue
19 priority_queue = [(0, start, None)]
20
21 while priority_queue:
22     weight, current, parent = heapq.heappop(priority_queue)
23
24     if current in visited:
25         continue
26

```

Output

```

Minimum Spanning Tree using Prim's Algorithm:

A - B : 2
B - C : 1
B - D : 4
D - E : 2

Total Weight of MST: 9

=== Code Execution Successful ===

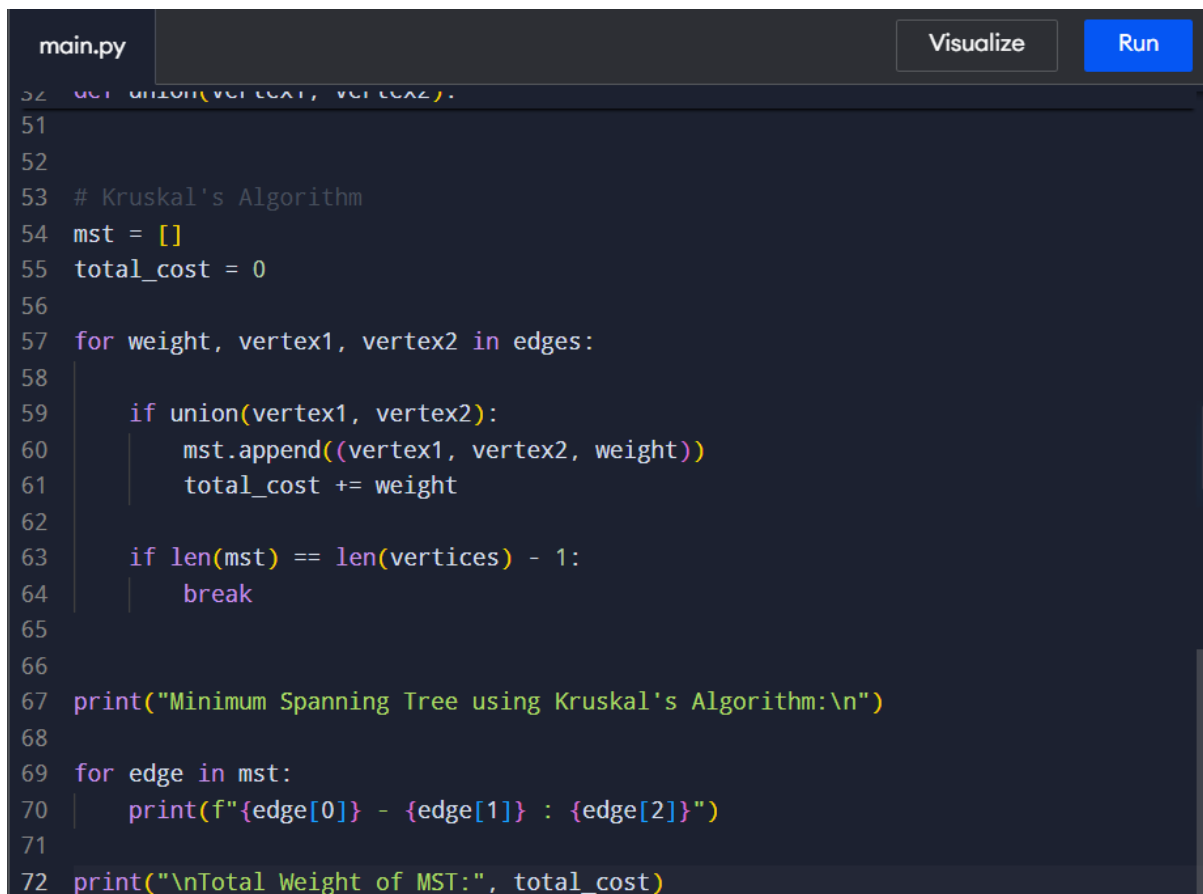
```

Kruskal's Algorithm – Steps

1. Start with all vertices as separate sets.
2. Sort all edges in increasing order of weight.

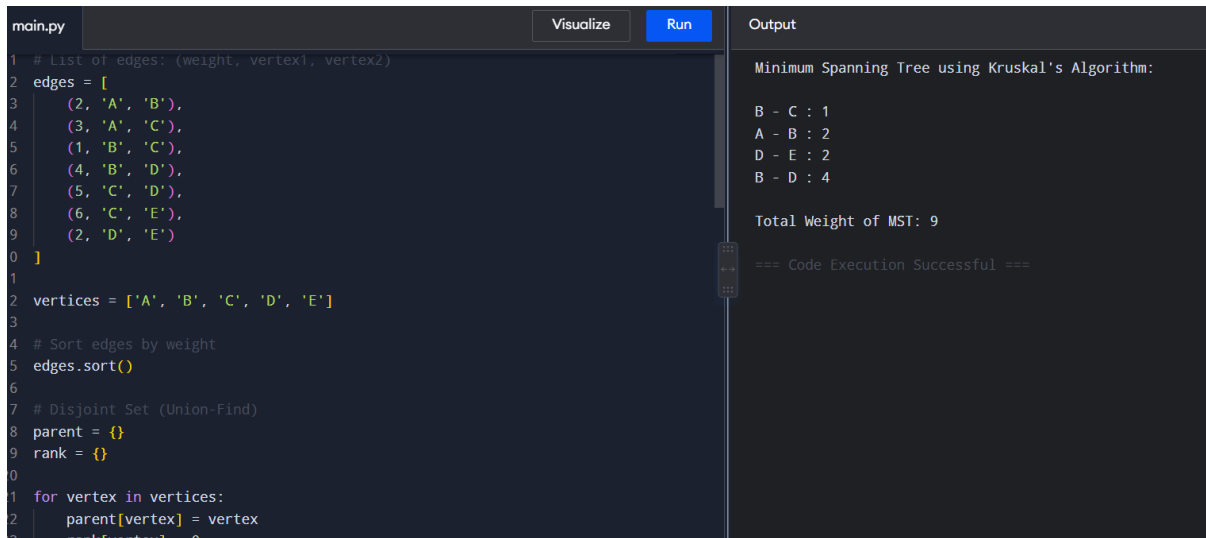
3. Select the edge with the minimum weight.
4. Check whether adding the edge creates a cycle.
5. If no cycle is formed, add the edge to the MST.
6. If a cycle is formed, reject the edge.
7. Repeat until the MST contains $V - 1$ edges.
8. Calculate the total weight of the MST.
9. Display the selected edges and total weight.
10. Stop.

Kruskal's Algorithm – Python Code

A screenshot of a Python IDE window titled 'main.py'. The code implements Kruskal's Algorithm. It starts with a function 'def union(vertex1, vertex2):' on line 50. Lines 51-52 are blank. Line 53 is a comment '# Kruskal's Algorithm'. Line 54 initializes 'mst = []'. Line 55 initializes 'total_cost = 0'. Line 56 is blank. Line 57 starts a 'for' loop 'for weight, vertex1, vertex2 in edges:'. Line 58 is blank. Line 59 has an 'if' statement 'if union(vertex1, vertex2):'. Line 60 indents 'mst.append((vertex1, vertex2, weight))'. Line 61 indents 'total_cost += weight'. Line 62 is blank. Line 63 has an 'if' statement 'if len(mst) == len(vertices) - 1:'. Line 64 indents 'break'. Line 65 is blank. Line 66 is blank. Line 67 prints 'Minimum Spanning Tree using Kruskal's Algorithm:\n'. Line 68 is blank. Line 69 starts a 'for' loop 'for edge in mst:'. Line 70 indents 'print(f"{edge[0]} - {edge[1]} : {edge[2]}")'. Line 71 is blank. Line 72 prints '\nTotal Weight of MST:', total_cost. The IDE has a 'Visualize' button and a 'Run' button in the top right corner.

```
50 def union(vertex1, vertex2):
51
52
53 # Kruskal's Algorithm
54 mst = []
55 total_cost = 0
56
57 for weight, vertex1, vertex2 in edges:
58
59     if union(vertex1, vertex2):
60         mst.append((vertex1, vertex2, weight))
61         total_cost += weight
62
63     if len(mst) == len(vertices) - 1:
64         break
65
66
67 print("Minimum Spanning Tree using Kruskal's Algorithm:\n")
68
69 for edge in mst:
70     print(f"{edge[0]} - {edge[1]} : {edge[2]}")
71
72 print("\nTotal Weight of MST:", total_cost)
```

Output



The screenshot shows a code editor with a file named 'main.py'. The code implements Kruskal's Algorithm to find a Minimum Spanning Tree (MST) for a graph with 5 vertices (A, B, C, D, E) and 9 edges. The edges are sorted by weight, and the algorithm uses a Disjoint Set (Union-Find) to avoid cycles. The output panel shows the results of the algorithm.

```
1 # List of edges: (weight, vertex1, vertex2)
2 edges = [
3     (2, 'A', 'B'),
4     (3, 'A', 'C'),
5     (1, 'B', 'C'),
6     (4, 'B', 'D'),
7     (5, 'C', 'D'),
8     (6, 'C', 'E'),
9     (2, 'D', 'E')
10 ]
11
12 vertices = ['A', 'B', 'C', 'D', 'E']
13
14 # Sort edges by weight
15 edges.sort()
16
17 # Disjoint Set (Union-Find)
18 parent = {}
19 rank = {}
20
21 for vertex in vertices:
22     parent[vertex] = vertex
23     rank[vertex] = 0
```

Output:

```
Minimum Spanning Tree using Kruskal's Algorithm:

B - C : 1
A - B : 2
D - E : 2
B - D : 4

Total Weight of MST: 9

=== Code Execution Successful ===
```

TIME COMPLEXITY

Prim's Algorithm

Using an adjacency matrix:

$O(V^2)$

Using a priority queue:

$O(E \log V)$

Prim's algorithm performs well for dense graphs.

Kruskal's Algorithm

The edges must first be sorted.

Therefore, the time complexity is:

$O(E \log E)$

Since:

$E \log E \approx E \log V$

Kruskal's algorithm is generally efficient for sparse graphs.

PERFORMANCE IN DENSE AND SPARSE GRAPHS

Dense Graphs

A dense graph contains a large number of edges.

Prim's algorithm is generally more suitable because it can efficiently grow the MST from connected vertices.

Sparse Graphs

A sparse graph contains relatively few edges.

Kruskal's algorithm is generally more suitable because it sorts and processes fewer edges.

TOOLS AND TECHNOLOGIES

The following tools can be used for implementation:

- Python – To implement Prim's and Kruskal's algorithms.
- Visual Studio Code – To write and execute the programs.
- NetworkX – For graph representation and analysis.
- Matplotlib – For graph visualization.

REAL-WORLD APPLICATIONS

Minimum Spanning Tree algorithms are used in:

- Network cable design.
- Telecom network planning.
- Road and railway network design.
- Electrical power distribution.
- Computer network optimization.

- Cluster analysis in data science.

ADVANTAGES

Prim's Algorithm

- Efficient for dense graphs.
- Builds the MST continuously.
- Easy to implement using priority queues.

Kruskal's Algorithm

- Efficient for sparse graphs.
- Simple edge-based approach.
- Uses Union-Find to avoid cycles.

CONCLUSION

Both Prim's and Kruskal's algorithms successfully construct the Minimum Spanning Tree using a greedy approach. Prim's algorithm builds the tree starting from a selected vertex, while Kruskal's algorithm selects the smallest available edges globally and avoids cycles. Prim's algorithm is generally more suitable for dense graphs, whereas Kruskal's algorithm is more suitable for sparse graphs. Therefore, the choice between Prim's and Kruskal's algorithms depends mainly on the structure and density of the graph.

RESULT

Thus, the Minimum Spanning Tree was successfully constructed using both Prim's and Kruskal's algorithms. Both algorithms produced the same minimum total weight of 9, while demonstrating different edge selection strategies and suitability for dense and sparse graphs.